

Introducción al desarrollo de interfaces



Índice

1. Interfaces multiplataformas
 - a. Reto del desarrollo multiplataformas
 - b. Herramientas y enfoques en el desarrollo
2. Arquitectura de CMP
 - a. ¿Que son KMP y CMP?
 - b. Targets de CMP
 - c. Impacto de los targets en el desarrollo de interfaces
3. Proyectos CMP
 - a. Estructura de módulos y carpetas
 - b. Mecanismo de Interoperabilidad
4. Paradigma declarativo y arquitectura
 - a. Del imperativo al declarativo
 - b. Ciclo de vida de la UI y los eventos
 - c. MVVM
5. Programando en CMP
 - a. Elementos básicos
 - b. Ejemplo base y proyecto base

Interfaces multiplataformas

Reto del Desarrollo Multiplataforma

Crear una aplicación para Android, iOS, Desktop y Web exige mantener 3 o 4 código-bases distintos.

Consecuencias:

- Multiplicación de costes y tiempos de desarrollo.
- Inconsistencias en la lógica de negocio entre plataformas.
- Equipos divididos (equipo Android, equipo iOS, equipo Web).

Interfaces multiplataformas

Herramientas y enfoques en el desarrollo

- **Nativo Puro:** Rendimiento máximo, pero 100% de duplicación de código.
- **Híbridos / Web-based (Cordova, Ionic):** Un solo código HTML/JS metido en un WebView. Rendimiento bajo y sensación poco nativa.
- **Renderizado Propio / Canvas (Flutter):** Dibuja sus propios widgets en un lienzo gráfico.
- **Puentes de Traducción (React Native):** Traduce componentes de JavaScript a nativo en tiempo de ejecución.

Interfaces multiplataformas

Herramientas y enfoques en el desarrollo

KMP permite decidir qué compartir. Puedes compartir solo la lógica de negocio (KMP puro) o compartir también la interfaz gráfica de forma declarativa mediante Compose Multiplatform (CMP).



Lógica de negocio



Interfaz gráfica



Arquitectura de CMP

¿Que son KMP y CMP?

- **Jetpack Compose:** El kit de herramientas de interfaz de usuario nativo de Google para crear diseños en Android.
- **Kotlin Multiplatform (KMP):** La tecnología base para compartir lógica de negocio (redes, bases de datos, modelos) entre Android, iOS, Desktop y Web sin incluir la interfaz.
- **Compose Multiplatform (CMP):** Framework creado por JetBrains que lleva Jetpack Compose más allá de Android para compartir la interfaz de usuario en múltiples plataformas (iOS, Desktop, Web).

Arquitectura de CMP

Targets de KMP

- **Android:**
 - **androidMain:** Compila a Bytecode JVM / DEX
- **Escritorio:**
 - **desktopMain:** Compila a JVM ejecutable en Windows, Linux y Mac.
- **Web**
 - **wasmJsMain:** Compila a WebAssembly para navegador.
 - **jsMain:** Compila a JavaScript para navegador.
- **Apple**
 - **iosMain:** Compila a binario nativo (LLVM / Framework iOS).

Arquitectura de CMP

Impacto de los targets en el desarrollo de interfaces

- **Factor de forma y entrada:**

- **Móvil:** Pantalla vertical, táctil, espacio reducido, gestos.
- **Escritorio:** Pantalla horizontal, ratón (hover/click secundario), teclado físico, multiventana.
- **Web:** Escalabilidad, comportamiento del navegador, eventos de scroll y cursor.

¿Deben compartirse los diseños entre todas las interfaces?

Proyectos de CMP

Estructura de módulos y carpetas

- **App:** Todo el contenido de la aplicación.
 - **AndroidApp:** Contenido específico del arranque de android.
 - **desktopApp:** Contenido específico del arranque de desktop.
 - **iosApp:** Contenido específico del arranque de IOS.
 - **Shared**
 - **AndroidMain:** Archivos específicos de lógica de negocio, gestión de datos o UI de android.
 - **commonMain:** Archivos de lógica de negocio, gestión de datos o UI común.
 - **iosMain:** Archivos específicos de lógica de negocio, gestión de datos o UI de IOS.
 - **jsMain:** Archivos específicos de lógica de negocio, gestión de datos o UI de web.
 - **jvmMain:** Archivos específicos de lógica de negocio, gestión de datos o UI de Escritorio.
 - **wasmJsMain:** Archivos específicos de lógica de negocio, gestión de datos o UI de WasmJs.
 - **webApp:** Contenido específico del arranque de web.
- **core:** Se guardan elementos comunes al núcleo de la app.
- **Gradle:** Carpeta para gestionar gradle y las versiones.
- **Server:** Módulo Backend de Ktor.

Proyectos de CMP

Mecanismo de Interoperabilidad

Generalmente en CMP tendremos el problema de que cada sistema usa funciones o formas diferentes de realizar las tareas de forma general esto se solucionara con mecanismos como el expect/actual, las interfaces y otros métodos que veremos durante el curso.

```
// En commonMain: Definimos QUÉ queremos hacer
expect fun getPlatformName(): String

// En androidMain: Definimos CÓMO se hace en Android
actual fun getPlatformName(): String = "Android " + android.os.Build.VERSION.RELEASE

// En desktopMain: Definimos CÓMO se hace en Desktop
actual fun getPlatformName(): String = "Desktop Java " + System.getProperty("java.version")
```

Paradigma Declarativo y Arquitectura Base

Del imperativo al declarativo

- **Imperativo:** Le decimos al sistema CÓMO cambiar la UI paso a paso (findViewById(), setText()). Propenso a fallos de sincronización.
- **Declarativo:** Describimos CÓMO debe verse la UI según el Estado actual. (Si los datos cambian la interfaz se redibuja)

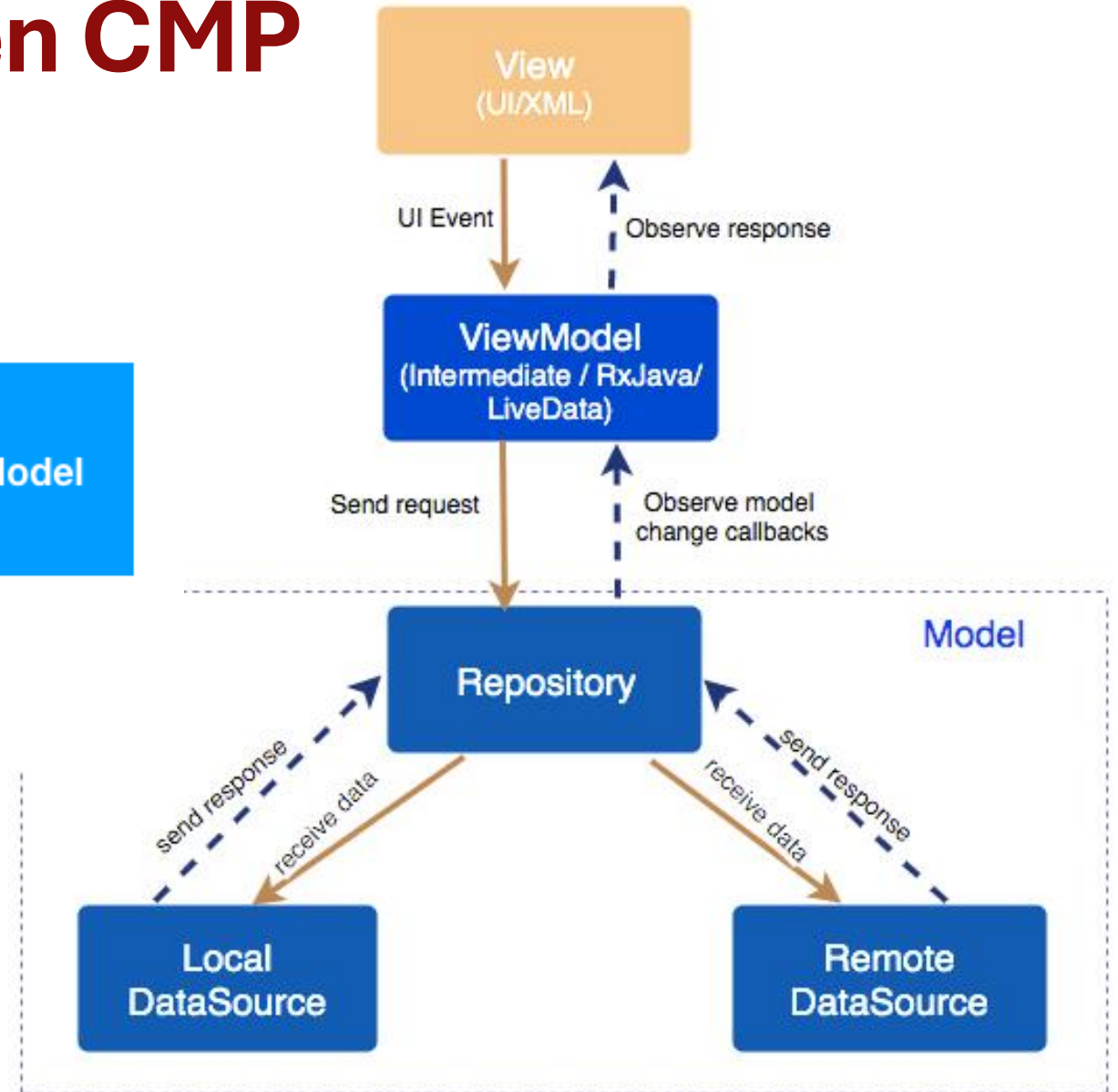
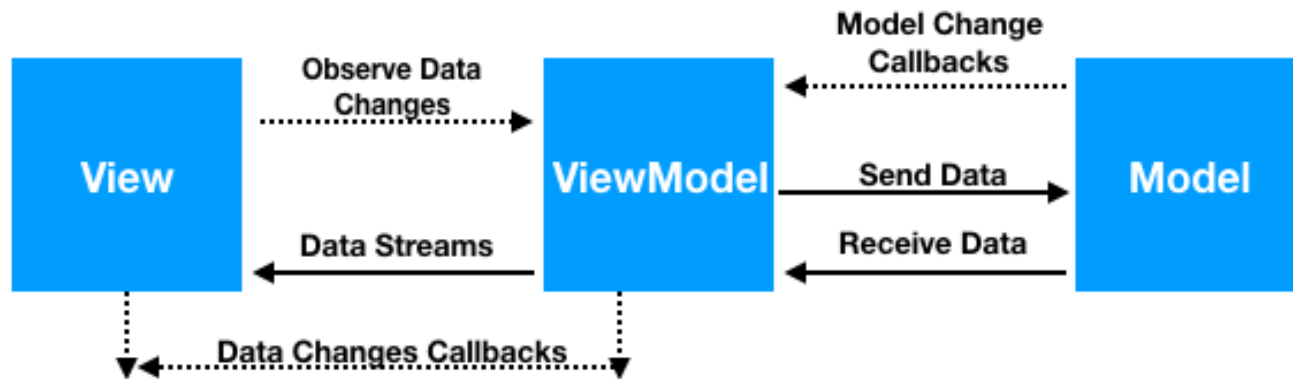
Paradigma Declarativo y Arquitectura Base

Ciclo de vida de la UI y los eventos

- **Composición:** Primera ejecución de la función @Composable donde se crea el árbol visual.
- **Recomposición:** Cuando una variable "observada" cambia, Compose re-ejecuta **únicamente** las partes de la pantalla afectadas por ese cambio.
- **Descomposición:** Se eliminan de la memoria los componentes que ya no deben mostrarse.

Arquitectura MVVM en CMP

MVVM



Programando en CMP

Elementos básicos

- **Componentes de entrada y texto:**
 - **Text("Hola"):** Muestra un texto en pantalla.
 - **TextField(...):** Campo de entrada de texto interactivo.
 - **Button(onClick = { ... }):** Botón interactivo.
- **Layouts de posicionamiento:**
 - **Column:** Apila elementos verticalmente.
 - **Row:** Coloca elementos horizontalmente.
 - **Box:** Superpone elementos unos encima de otros.

Programando en CMP

Ejemplo base y proyecto base